

Doc. no.:	P0963R0
Date:	2018-02-05
Audience:	Evolution Working Group
Reply-to:	Zhihao Yuan <zy at miator dot net>

Structured binding declaration as a *condition*

Motivation

```
if (auto [ok, value] = readint(...); ok == format_errc::no_error)
```

is not as clear and simple as

```
if (auto [ok, value] = readint(...))
```

when

```
if (readint(...))
```

is starting a valid `if` statement.

Discussion

C++17 structured binding declaration is designed as a variant of variable declarations. More specifically, it is a variant to *simple-declaration* and *for-range-declaration*. However, the grammar production *condition* is, conditionally, also a kind of variable declaration,

```
if (auto rc = readint(...))
```

so there is little reason not to allow the above to be replaced by

```
if (auto [ok, value] = readint(...))
```

when the return type qualifies contextually convertible to `bool` and decomposable at the same time.

In a word, we consider the status quo as a wording oversight, presumably affected by the fact that there is no BNF production named “condition-declaration” in the standard text. Therefore we suggest applying the proposed change as a DR against C++17.

Design Decisions

It is tempting to add extra semantics given the proposed syntax, such as making the decomposition work (accessing `e.get()`, for example) conditional when the underlying object for decomposition, `e`, evaluates to `false` after contextually converted to `bool`. It may enable more programming styles such as throwing `get()`. The author considers that this idea complicates the mental model for using the two features together, and decided not to include it in this paper.

It is worthwhile to figure out what array decomposition does in *condition*. *condition* bans declaring arrays, so this paper does not allow decomposing array `e` either. However, *condition* accepts array references, which always evaluate to `true`, and this is also unchanged in this paper. That is,

```
if (auto& [a, b, c] = "ht")
```

works with the proposed change.

Technically we can enable decomposing arrays in *conditions*, but the author is not motivated to evaluate.

Wording

The wording is relative to N4713.

Extend the grammar in 9 [stmt.stmt]/1 as follows:

condition:

expression

*attribute-specifier-seq*_{opt} *decl-specifier-seq* *declarator* *brace-or-equal-initializer*

*attribute-specifier-seq*_{opt} *decl-specifier-seq* *ref-qualifier*_{opt} [*identifier-list*]

brace-or-equal-initializer

Modify 9 [stmt.stmt]/2 as follows:

The rules for *conditions* apply both to selection-statements and to the `for` and `while` statements (9.5). The *declarator* shall not specify a function or an array. The *decl-specifier-seq* shall not define a class or enumeration. If the *auto type-specifier* appears in the *decl-specifier-seq*, the type of the identifier being declared is deduced from the initializer as described in 10.1.7.4. If *identifier-list* appears in the *condition*, the declaration is a structured binding declaration (11.5), where the *assignment-expression* in the *brace-or-equal-initializer* shall not have array type if no *ref-qualifier* is present.

Modify paragraph 3 as follows:

A name introduced by a declaration in a *condition* (~~either~~may be introduced by the *decl-specifier-seq*, ~~or the *declarator*, or the *identifier-list*~~ of the condition) is in scope from its point of declaration until the end of the substatements controlled by the condition. If the name is redeclared in the outermost block of a substatement controlled by the condition, the declaration that redeclares the name is ill-formed. [*Example:*

```
if (int x = f()) {  
    int x;           // ill-formed, redeclaration of x  
}  
else {  
    int x;           // ill-formed, redeclaration of x  
}
```

```
if (auto& [a] = "") {  
    char a;           // ill-formed, redeclaration of a  
}
```

–end example]

Insert a paragraph between paragraph 3 and 4 in 9 [stmt.stmt]:

The variable of a *condition* that is an initialized declaration is the declared variable. The variable of a *condition* that is a structured binding declaration (11.5) is the variable `e` with a unique name.

Rewrite the original paragraph 4 as follows:

If a *condition* is an expression, the value of the condition is the value of the expression, contextually converted to `bool` for statements other than `switch` ; if that conversion is ill-formed, the program is ill-formed. Otherwise, in a `switch` statement, the value of the condition is the value of the variable of the *condition* if it has integral or enumeration type, or of that variable implicitly converted to integral or enumeration type otherwise. In a statement other than a `switch` statement, the value of the condition is the value of the variable of the *condition* contextually converted to `bool` (Clause 7). If that conversion is ill-formed, the program is ill-formed. The value of the condition will be referred to as simply “the condition” where the usage is unambiguous.

Implementation

The proposed change has been implemented in Clang 6.0.0 guarded by `-Wbinding-in-condition` : <https://godbolt.org/g/hgefUz> (<https://godbolt.org/g/hgefUz>)

The implementation is a one-line-change without counting in diagnosis and tests, giving a sense of how natural the proposed change is.

Acknowledgement

Thank Richard Smith for encouraging the work.